

OptiCine: Constraint-Based Cinema Resource Scheduling with Versioned Approval Workflows

Le Ngoc Dang, Hoang Tan Thanh, Minh Tuan Nguyen, Nguyen Thai, and Vinh Le

FPT University, Da Nang, Vietnam

Abstract. Cinema showtime planning is a constrained resource-allocation problem in which movies, screening rooms, maintenance intervals, cleaning buffers, and business hours must be coordinated without operational conflicts. Manual planning through spreadsheets and informal communication is difficult to scale and provides limited traceability when schedules are reviewed or revised. This paper presents **OptiCine**, a research-oriented design for an internal cinema operations and scheduling system. The scheduling problem is formulated as a Constraint Satisfaction Problem (CSP) with hard feasibility constraints and configurable utility objectives. A version-first domain model separates a logical **Schedule** from its concrete **ScheduleVersion** records, allowing approval, publication, disruption handling, and historical comparison without overwriting earlier plans. The proposed architecture uses the Strategy pattern for replaceable scheduling objectives, the Chain of Responsibility pattern for two-level approval, and the Builder pattern for constructing schedule versions and their showtimes. The paper also specifies database integrity, authorization, audit, and one-way publication requirements. The result is a coherent design basis for future implementation and evaluation; no production deployment or measured performance is claimed.

Keywords: Cinema scheduling · Constraint Satisfaction Problem · Resource allocation · Schedule versioning · Approval workflow · Software design patterns

1 Introduction

Cinema branches must coordinate a continuously changing set of movies, screening rooms, operating hours, maintenance periods, and audience-demand assumptions. A valid daily or weekly plan must avoid room overlap, preserve a cleaning interval between consecutive screenings, exclude unavailable rooms, respect business hours, and assign suitable room types. The planning problem becomes more difficult when managers also want high-popularity movies in larger rooms or high-weight time slots, while maintaining content diversity and limiting idle gaps.

Many small and medium cinema branches still prepare showtime plans through spreadsheets, email, and chat groups. These tools are familiar but do not provide a shared constraint model, a controlled approval lifecycle, or a reliable history of

revisions. A change made after review may overwrite the previous plan, and a room incident may require staff to reconstruct affected showtimes manually. The problem is therefore not only schedule generation; it also includes governance, traceability, and safe publication.

OptiCine is proposed as an internal cinema operations and resource-scheduling system. Its scope includes movie, room, maintenance, showtime, schedule, approval, notification, audit, and projected-analytics functions. Customer ticket booking, seat selection, payment, refund processing, loyalty programmes, and real-time ticket inventory remain outside the system. A future ticketing platform is treated only as a downstream consumer of published schedule data and schedule-change notifications.

The design models generation as a CSP because feasibility is determined by multiple interacting rules. Configurable strategies can then rank feasible candidates according to utility, diversity, or stability. Schedule changes are managed through **ScheduleVersion**: each version owns its own set of showtimes and is reviewed through explicit approval requests. The workflow contains two levels, Shift Supervisor and Branch Manager, before publication. A returned version follows a revision path, whereas a rejected version is closed and cannot return directly to draft.

The main contributions of this work are:

- a cinema-specific constraint model covering overlap, cleaning buffer, maintenance, business hours, active resources, and room compatibility;
- a version-first schedule model that preserves approval and disruption history without editing submitted or published content in place;
- a modular architecture using Strategy, Chain of Responsibility, and Builder patterns to separate scheduling objectives, approval levels, and complex version construction; and
- explicit data-integrity, role-based authorization, audit, and one-way publication rules that make the proposed design traceable from requirements to persistence.

The remainder of the paper reviews relevant scheduling research, defines the OptiCine domain and constraints, describes the proposed architecture and workflow, and discusses design validation, limitations, and future work.

2 Related Work

Automated scheduling research spans metaheuristic optimization, real-time resource allocation, hybrid learning frameworks, and disruption recovery. Although these studies are not cinema-specific, they provide useful search, evaluation, and stability concepts for the proposed system.

2.1 Metaheuristic and Multi-Objective Scheduling

Selvaraj et al. applied Crow Search Algorithm to optimal power scheduling and showed how a population-based search can balance exploration and exploitation

under operational constraints [9]. Du et al. formulated DAG-structured workload scheduling and proposed a whale-optimization approach for coordinating dependent tasks across computing resources [1]. Zhao combined genetic algorithms and particle swarm optimization for multi-objective power-system scheduling [12]. These works motivate treating scheduling objectives as replaceable strategies rather than embedding one algorithm into the domain model.

2.2 Real-Time and Hybrid Frameworks

Liu et al. proposed a hybrid framework that combines federated reinforcement learning with PSO-GA for real-time and non-real-time fog-computing tasks [5]. Naeem et al. studied intelligent job scheduling and resource optimization in an edge-cloud continuum [6], while Qi et al. used deep reinforcement learning for real-time digital-twin task scheduling in cloud environments [7]. These studies highlight the need to separate compute-intensive scheduling from responsive operational views, but OptiCine does not claim to implement their algorithms.

2.3 Disruption and Benchmarking

Xiang et al. used multi-agent reinforcement learning for open-pit mine truck coordination [11]. Stadtler et al. developed a benchmark for selecting real-time scheduling methods for automated guided vehicle fleets [10]. Li et al. proposed inverse scheduling for aircraft assembly, minimizing disruption when an existing plan becomes infeasible [4]. The latter is particularly relevant to cinema incidents: a room failure should move as few published showtimes as possible rather than regenerate an unrelated plan from scratch.

2.4 Research Gap

The reviewed work addresses complex scheduling domains but does not combine the cinema-specific constraints considered here with schedule versioning, two-level approval, and controlled publication. OptiCine fills this design gap by combining a CSP formulation [8] with established software patterns [3,2]. The contribution is a coherent, research-oriented system design rather than a claim of algorithmic superiority or measured production performance.

3 OptiCine Domain and Scope

3.1 Actors and Responsibilities

OptiCine defines four internal roles. Cinema Staff maintain resource data, generate and edit draft schedules, submit versions, and record incidents. Shift Supervisors perform level-one operational review. Branch Managers perform level-two business review, publish approved versions, and inspect projected analytics. System Administrators manage accounts, roles, permissions, configuration, and audit access.

Table 1. Core entities and responsibilities.

Entity	Responsibility
Schedule	Logical daily or weekly planning period.
ScheduleVersion	Concrete versioned plan, strategy snapshot, workflow state, and projected metrics.
Showtime	Movie-room-time assignment owned by one schedule version.
Movie	Reusable movie data, duration, popularity input, and operational status.
Room	Physical screening resource, capacity, type, and persistent status.
RoomMaintenance	Planned or emergency room-unavailability interval.
ApprovalRequest	Review assignment for one version and one workflow step.
ApprovalComment	Reviewer comment, approval, return explanation, or rejection reason.

3.2 Core Domain Model

A **Schedule** identifies a daily or weekly planning period. A **ScheduleVersion** stores one concrete plan for that period, including its strategy snapshot, utility score, projected metrics, status, and owned **Showtime** records. A showtime references exactly one movie, one room, and one schedule version. **ApprovalRequest** and **ApprovalComment** records reference the exact version under review. **RoomMaintenance** records represent planned or emergency intervals in which a room cannot be used.

This separation prevents a common traceability defect: showtimes and approval records do not point directly to the logical schedule. Instead, they point to the concrete version. Consequently, a returned, rejected, published, or archived plan remains available even after a new draft is created.

4 Constraint-Based Scheduling Model

4.1 Inputs and Decision Variables

Let M be the set of active movies, R the set of active rooms, D the dates in the planning period, and T the candidate start times. The binary decision variable

$$x_{mrdt} = \begin{cases} 1, & \text{if movie } m \text{ starts in room } r \text{ on date } d \text{ at time } t, \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

represents a candidate showtime assignment. Inputs include movie duration and popularity, room capacity and type, maintenance intervals, business hours, cleaning buffer, time-slot weights, strategy parameters, and optional show count limits.

4.2 Hard Constraints

A candidate schedule is feasible only when all hard constraints are satisfied. The principal rules are summarized in Table 2.

Table 2. Hard scheduling constraints.

ID	Constraint
HC-01	A room hosts at most one showtime at any instant.
HC-02	The next room use starts after the previous end time plus the configured cleaning buffer.
HC-03	The room is ACTIVE and the occupied interval does not overlap maintenance.
HC-04	The movie is ACTIVE for the scheduled date.
HC-05	Start and end times remain within configured business hours.
HC-06	End time equals start time plus movie duration; cleaning buffer is evaluated separately.
HC-07	Room type and configured movie restrictions are compatible.
HC-08	Submitted, approved, published, and archived versions are not edited in place.

For two showtimes a and b in the same room, the occupied intervals include the cleaning buffer. They must satisfy

$$e_a + b_a \leq s_b \quad \vee \quad e_b + b_b \leq s_a, \quad (2)$$

where s , e , and b denote start time, end time, and buffer. Maintenance intervals are checked using the same half-open interval interpretation.

4.3 Utility and Projected Revenue

After feasibility validation, a strategy may rank candidates using the simple utility function

$$U(x) = \sum_{m,r,d,t} x_{mrdt} p_m c_r w_t - \lambda_1 G(x) - \lambda_2 \Delta(x), \quad (3)$$

where p_m is a normalized popularity input, c_r is room capacity, w_t is a time-slot weight, $G(x)$ penalizes avoidable idle gaps, and $\Delta(x)$ penalizes changes from a published version during rescheduling. U is a schedule utility score, not actual revenue.

Projected revenue is estimated separately as

$$R_{\text{projected}}(x) = \sum_{m,r,d,t} x_{mrdt} \hat{o}_{mrdt} c_r \bar{v}, \quad (4)$$

where $\hat{o}_{mrdt}$ is projected occupancy and $\bar{v}$ is a configured average ticket value. Actual ticket sales remain outside OptiCine.

4.4 Schedule Generation Workflow

Figure 1 shows the proposed generation flow. Staff select a period, allowed resources, and a strategy. The system loads active movies, available rooms,

maintenance intervals, business rules, buffers, and weights. The selected strategy generates candidate assignments, after which all hard constraints are validated. A feasible candidate is saved as a draft schedule version; otherwise, the system records constraint violations and returns a report for parameter adjustment. Submission is a separate user decision and creates the first approval request only after final validation.

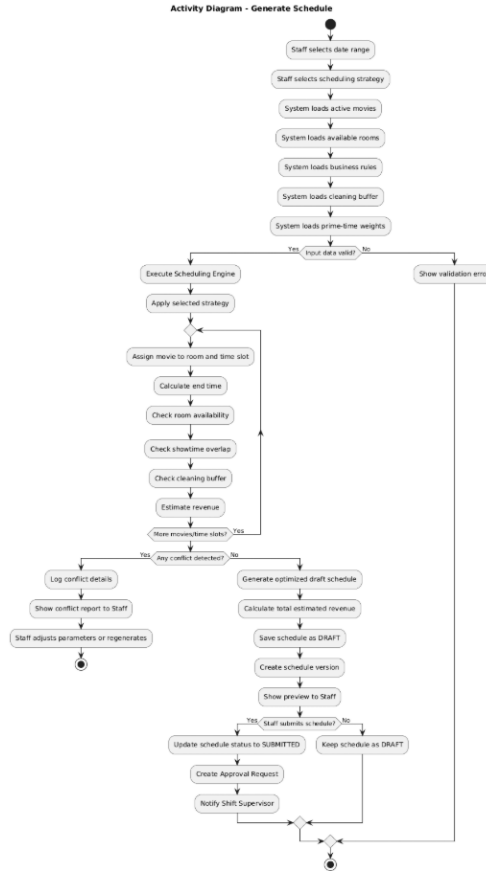


Fig. 1. Proposed activity flow for schedule generation and optional submission.

5 Architecture and Design Patterns

5.1 Layered Modular Architecture

The proposed architecture separates presentation, application coordination, domain rules, and infrastructure adapters. Presentation components provide calendar, approval, dashboard, and administration views. Application services coordi-

nate use cases such as generation, submission, review, publication, and incident handling. The domain layer contains schedule versioning, constraint validation, strategies, workflow transitions, and projected metrics. Infrastructure adapters provide relational persistence, optional availability caching, notifications, file storage, audit, and one-way publication to a future ticketing platform.

Technology names such as Spring Boot, MySQL, or Redis are possible implementation choices rather than mandatory properties of the model. The design requires stable interfaces so that persistence, caching, notification, and publication mechanisms can be replaced without modifying the core domain.

5.2 Strategy Pattern

The Strategy pattern isolates candidate generation and evaluation behind a common **SchedulingStrategy** interface [3]. A constraint-first baseline prioritizes deterministic feasibility. A revenue-oriented strategy emphasizes popularity, room capacity, and time-slot weight. A diversity-oriented strategy balances movie exposure across rooms and periods. A stability-oriented strategy minimizes moved or cancelled showtimes during disruption. More advanced algorithms, including GA-PSO or other metaheuristics, can be evaluated later without changing the workflow or data model.

5.3 Builder Pattern

A schedule version is a complex aggregate containing many showtimes, configuration snapshots, buffers, utility, and projected metrics. The Builder pattern separates incremental construction from generation coordination. A builder initializes the planning period and strategy, adds validated showtimes, records the applicable configuration, calculates metrics, and returns a complete **ScheduleVersion**. This avoids exposing partially constructed versions to approval or publication.

5.4 Chain of Responsibility and Version State

Approval is modeled as a two-level chain: Shift Supervisor followed by Branch Manager. Each handler validates authorization and the expected workflow state. An approval at level one changes the version to **SUPERVISOR_APPROVED** and creates a manager request. Approval at level two changes the version to **MANAGER_APPROVED**, making it eligible for publication.

Return and rejection are distinct. A return moves the reviewed plan back to a draft revision path and requires an explanatory comment. A rejection changes the reviewed version to **REJECTED**; it cannot return directly to draft, and further work requires a new draft version. Figure 2 summarizes the controlled lifecycle.

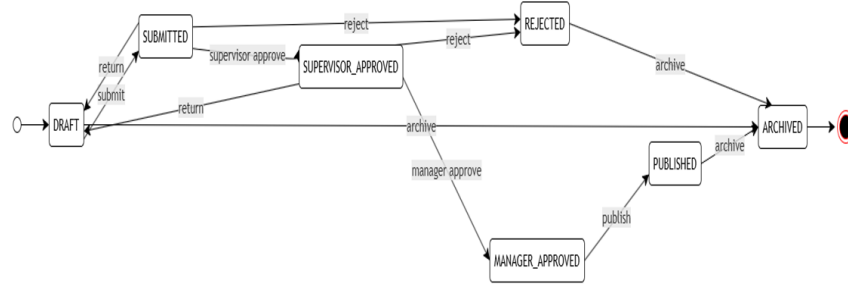


Fig. 2. ScheduleVersion lifecycle with distinct return and rejection paths.

6 Publication, Disruption, and Data Integrity

6.1 Controlled Publication

Only a `MANAGER_APPROVED` version may be published. Conceptually, the publication transaction locks the logical schedule, verifies eligibility, archives a previously published version when applicable, marks the selected version and its showtimes as `PUBLISHED`, records an audit event, and creates notification or delivery records. The state change is atomic: a failure must not leave a partially published plan.

External ticketing integration is one-way. OptiCine sends published schedule data and change notifications but does not modify customer orders, tickets, payments, or refunds. Delivery failure is recorded for retry without rolling back a valid internal publication.

6.2 Disruption and Rescheduling

When a room incident affects published showtimes, staff record the unavailable interval and the system identifies impacted assignments. A new schedule version is copied from the official plan. The stability-oriented strategy searches alternative rooms and times while minimizing changes. A feasible revision is saved as draft, follows an expedited approval path, and is published as a new official version. If no feasible assignment exists, unresolved showtimes are presented for manual adjustment or cancellation, and the Branch Manager records the decision. Refund processing remains outside the system.

6.3 Relational Integrity and Indexing

The relational design enforces `UNIQUE(schedule_id, version_number)`. Each showtime references one schedule version, movie, and room. Approval requests reference the reviewed version rather than the logical schedule. At most one active `PENDING` approval request exists for a version and workflow step, while historical decisions are retained.

The proposed indexes support the principal access paths:

- `showtimes(room_id, start_time, end_time)` for room-conflict checks;
- `showtimes(schedule_version_id)` for loading a complete version;
- `room_maintenances(room_id, start_time, end_time)` for maintenance overlap; and
- `approval_requests(schedule_version_id, current_step, status)` for workflow queues.

Audit records are append-only. Historical schedule versions, showtimes, approval comments, and publication events are preserved for comparison and traceability.

7 Design Validation and Discussion

The proposed design is validated qualitatively through requirement traceability and scenario walkthroughs rather than execution benchmarks. Three representative scenarios cover the principal risks.

Normal generation and publication. Staff request a weekly schedule, the engine creates and validates candidates, a draft version is stored, both approval levels accept it, and the manager publishes it. The scenario traces from constraints to version ownership, approval requests, atomic publication, and downstream notification.

Return versus rejection. A supervisor may return a submitted version to a draft revision path, whereas a rejection closes the reviewed version. The distinction preserves decision history and prevents a rejected record from being silently edited into a different plan.

Room disruption. An incident marks a maintenance interval, identifies published showtimes, creates a new version, and searches for minimal changes. The previous official version remains in history, and unresolved showtimes are escalated without introducing refund or ticket-order responsibilities.

The design has several limitations. It assumes one cinema branch, relies on configured or predicted popularity and occupancy inputs, and does not specify one implemented solver. No latency, optimality, or revenue-improvement results are reported. Future implementation should compare deterministic, metaheuristic, and hybrid strategies using feasibility rate, computation time, utility, stability, and sensitivity to demand-estimation error.

8 Conclusion

This paper presented OptiCine, a research-oriented design for internal cinema resource scheduling. The system formulates showtime planning as a CSP, separates feasibility from configurable utility objectives, and distinguishes projected analytics from actual ticket revenue. Its core contribution is a version-first workflow in which showtimes and approval records belong to a specific `ScheduleVersion`. This structure supports controlled return, rejection, publication, disruption recovery, historical comparison, and audit without overwriting prior plans.

Strategy, Builder, and Chain of Responsibility patterns provide explicit extension points for scheduling objectives, aggregate construction, and approval levels. Database constraints, atomic publication, role-based access, and append-only audit rules complete the proposed design. Future work should implement and benchmark candidate scheduling strategies, evaluate occupancy prediction, and validate one-way ticketing integration while keeping customer transactions outside the operational core.

References

1. Du, N., Ji, Y., Wu, C., Hou, A., Nie, W.: Scheduling DAG-structured workloads based on whale optimization algorithm. *The Journal of Supercomputing* **81**, 911 (2025). <https://doi.org/10.1007/s11227-025-07415-3>
2. Fowler, M.: *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA (2002)
3. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA (1994)
4. Li, T., Hua, M., Wang, J., Qin, W.: Inverse scheduling method for aircraft flat-tail assembly production based on improved genetic algorithm. *Scientific Reports* **15**, 23898 (2025)
5. Liu, F., Liu, Z., Liu, X., Zhou, H.: Scalable hybrid framework for real time and non real time task scheduling in fog computing using federated reinforcement learning and PSO-GA. *Scientific Reports* **15**, 22218 (2025)
6. Naeem, A.B., Senapati, B., Rasheed, J., Baili, J., Osman, O.: An intelligent job scheduling and real-time resource optimization for edge-cloud continuum in next generation networks. *Scientific Reports* **15**, 25452 (2025)
7. Qi, D., et al.: Real-time scheduling of power grid digital twin tasks in cloud via deep reinforcement learning. *Journal of Cloud Computing* **13**, 121 (2024). <https://doi.org/10.1186/s13677-024-00683-z>
8. Russell, S., Norvig, P.: *Artificial intelligence: A modern approach* (chapter 6: Constraint satisfaction problems). Prentice Hall (2010)
9. Selvaraj, G., Rajangam, K., Vishnuram, P., Bajaj, M., Zaitsev, I.: Optimal power scheduling in real-time distribution systems using crow search algorithm for enhanced microgrid performance. *Scientific Reports* **14**, 82061 (2024). <https://doi.org/10.1038/s41598-024-82061-y>
10. Stadtler, J., Stahmann, P., Janiesch, C.: A benchmark for selecting real-time scheduling approaches for fleets of automated guided vehicles in intralogistics. *Business & Information Systems Engineering* (2026). <https://doi.org/10.1007/s12599-026-01003-2>
11. Xiang, X., Lin, W., Li, D.: A hybrid MARL clustering framework for real time open pit mine truck scheduling. *Scientific Reports* **15**, 16347 (2025)
12. Zhao, X.: Research on multi-objective scheduling optimization of power system based on hybrid model of genetic algorithm and particle swarm optimization. *Discover Computing* **29**, 209 (2026). <https://doi.org/10.1007/s10791-026-09929-7>